stacked

Stacked cubes and dices

v0.2.0 Akilon <https://github.com/Akilon27> MIT

A propos

Les empilements de cubes sont apparus dans les programmes de collège en 2025, d'où ce paquet.

Je me suis inspiré de `tikz3d-fr` (auteur : Cédric Pierquet) et d'une partie de `ProfCollege` de LaTeX (auteur : Christophe Poulain) mais l'implémentation n'en est pas une copie. J'ai utilisé l'IA (Gemini et Claude) pour les dés et la partie aléatoire de la fonction `pile`, le reste est de moi sauf la fonction `dice-face` créée par eric1102 (sur Discord), inclue ici avec son accord.

La version 2 ajoute de nombreuses fonctions pour représenter des solides (polyèdres, cônes, cylindres, sphères), leurs sections par un plan (défini soit par 3 points soit par les 4 coefficients de l'équation cartésienne), leurs patrons. Des fonctions d'aide pour avoir automatiquement les coordonnées des sommets sont définies pour certains cas classiques (pyramides, prismes et quelques polyèdres réguliers : tétraèdre, cube, octaèdre, dodécaèdre et icosaèdre). De plus une fonction globe permet d'expliquer les coordonnées géographiques/sphériques.

Fonctions v1 : « piles » de cubes et dés

- cube()
- pile()
- building()
- above-view()
- right-view()
- front-view()
- below-view()
- left-view()
- back-view()
- maincoords()
- dice()
- throws()
- dice-face()

cube

Commande cetz pour un cube

Paramètres

```
cube(
  a: array,
  c: auto color array,
  border-stroke: boolean color,
  thickness: int float ratio,
  light: float ratio array,
  side: boolean
) -> function
```

a `array`

Position

c `auto` or `color` or `array`

auto → (rouge à droite, vert en haut, jaune à gauche), sinon une ou 3 couleurs

Par défaut: `auto`

border-stroke `boolean` or `color`

Lignes en noir ou de la couleur du cube, défaut false

Par défaut: `false`

thickness `int` or `float` or `ratio`

Epaisseur des tracés (de 1pt), défaut 100%

Par défaut: `100%`

light `float` or `ratio` or `array`

De combien éclaircir quand une seule couleur 20% par défaut

Par défaut: `20%`

side `boolean`

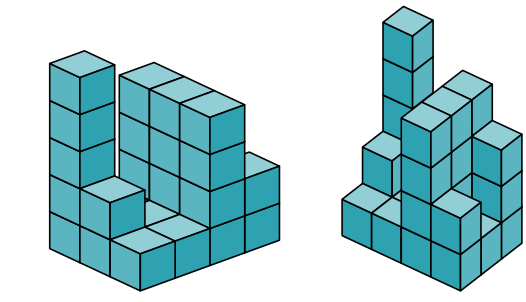
Commande interne pour la vue de côté avec `pile`

Par défaut: `false`

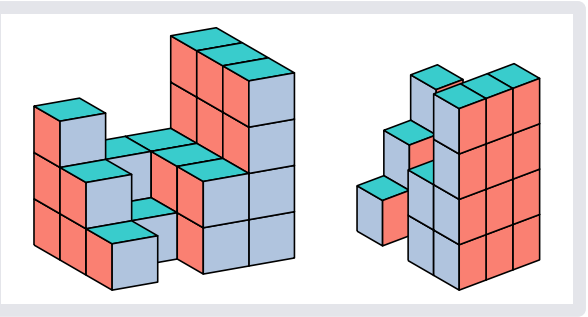
pile

Empilement de petits cubes sur une grille, donnés sous forme de liste allant du fond vers l'avant `3251` puis de gauche à droite (séparés par des virgules), une couleur peut être indiquée au début ou à la fin, couleurs de base avec none : avant → jaune, dessus → vert et droite → rouge, sinon une liste de 3 couleurs peut-être donnée dans colors

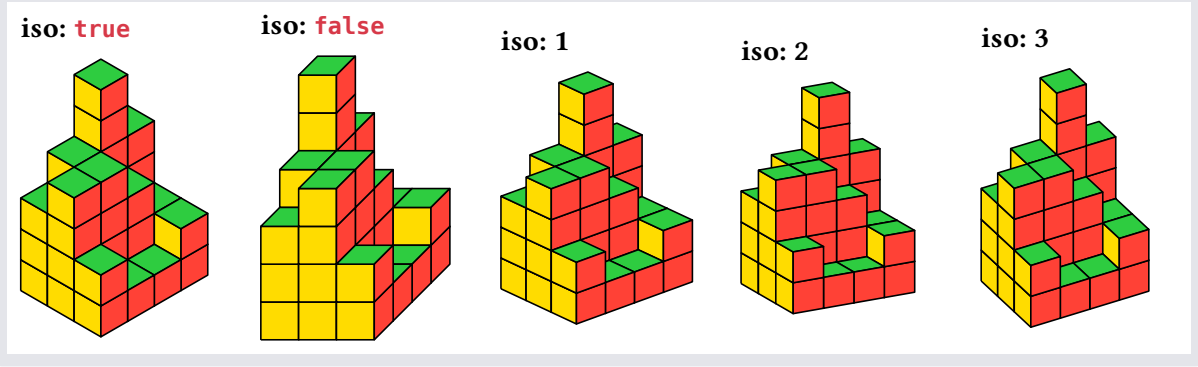
```
#pile(3415,1412,2411,eastern,side:true,
iso:1,draw-scale:.6,border-stroke:true)
```



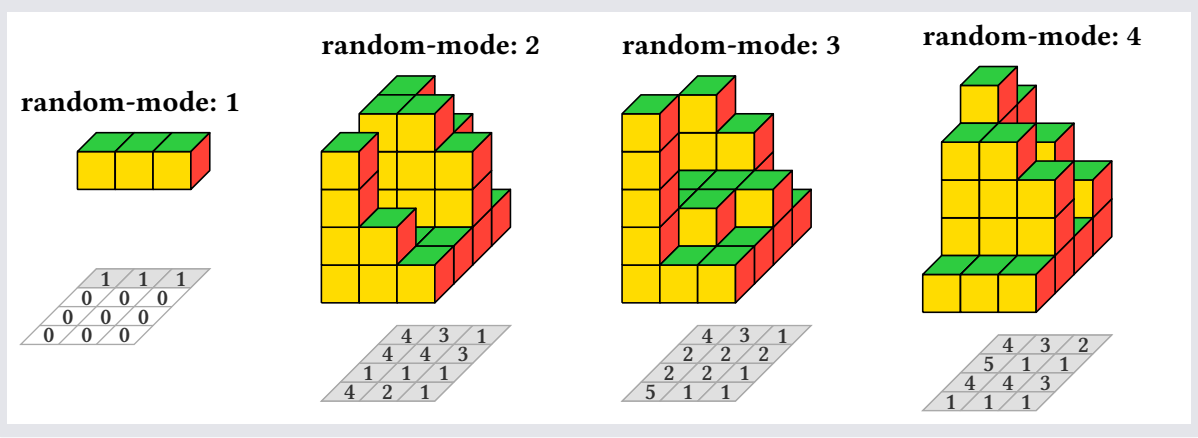
```
#pile(4223,4212,4201,colors:
(rgb("#fa8072"),rgb("#b0c4de"),teal),
side: true,iso: 2,draw-scale: .75)
```



```
#grid(columns: 5*(1fr,), column-gutter: 1.5em,
 ..for i in (true,false,1,2,3) {([=== iso: #i
   #pile("4643",2344,2112, iso: i)
 ],)}
 )
```




```
#grid(columns: 5*(3.2cm,), column-gutter: 2em,
 ..for i in (1,2,3,4) {([=== random-mode: #i
   #pile(random:true, random-mode: i,shadow:true,seed:2*i,plan:true)
 ],)}
 )
```



`z:0` pour vue de face `y:0, z:(0,-1)` pour vue du dessus `x:0, z:(-1,0)` pour vue de droite

Paramètres

```
pile(
  ..a: int str array arguments,
  random: boolean,
  width: int,
  depth: int,
  height: int,
  seed: int,
  plan: boolean,
  labels: boolean,
  x: array,
  y: array,
  z: array,
  thickness: int float ratio,
  border-stroke: boolean color,
  side: boolean,
  light: float ratio array,
  draw-scale: float ratio,
  x-spread: float int,
  y-spread: float int,
  z-spread: float int,
  colors: array,
  grids: boolean,
  solution: boolean,
  below-separation: auto int float,
  left-separation: auto int float,
  back-separation: auto int float,
  iso: boolean int,
  space: length relative length fraction,
  holes: boolean,
  random-mode: auto int,
  views: array,
  views-names: array,
  axes: boolean,
  opacity: ratio,
  shadow: boolean,
  disposition: str,
  box-args: dictionary
) -> content
```

..a `int` or `str` or `array` or `arguments`

Liste de nombres/string correspondants aux hauteurs de l'arrière vers l'avant, de gauche à droite

random `boolean`

Active un mode aléatoire : génère automatiquement l'empilement (au lieu des arguments positionnels), avec des hauteurs égales ou décroissantes de rang en rang

Par défaut: `false`

width `int`

Largeur (nombre de tranches selon X) en mode aléatoire

Par défaut: `3`

depth `int`

Profondeur (nombre de chiffres par tranche selon Z) en mode aléatoire

Par défaut: `4`

height `int`

Hauteur maximale (selon Y) en mode aléatoire

Par défaut: `5`

seed `int`

Graine du générateur pseudo-aléatoire pour la reproductibilité du tirage : même graine → même empilement, à changer pour en obtenir un autre

Par défaut: `1`

plan `boolean`

Affiche le plan de niveau (vue de dessus chiffrée)

Par défaut: `false`

labels `boolean`

Affiche les intitulés des vues sur les grids

Par défaut: `false`

x `array`

Direction de la première coordonnée, défaut (1,0)

Par défaut: `(1,0)`

y `array`

Direction de la seconde coordonnée, défaut (0,1)

Par défaut: `(0,1)`

z `array`

Direction de la troisième coordonnée, défaut (-0.5,-0.5)

Par défaut: `(-.5,-.5)`

thickness `int` or `float` or `ratio`

Epaisseur des tracés (de 1pt), défaut 60% soit 0.6pt

Par défaut: `60%`

border-stroke `boolean` or `color`

Lignes en noir, défaut false

Par défaut: `false`

side `boolean`

Ajout d'une seconde vue de l'assemblage depuis la « droite »

Par défaut: `false`

light `float` or `ratio` or `array`

De combien éclaircir quand une seule couleur

Par défaut: `20%`

draw-scale `float` or `ratio`

Echelle du dessin : modifie la taille des arêtes d'un cube de base

Par défaut: `.5`

x-spread `float` or `int`

Permet « d'éclater » l'empilement construit vers la droite afin de mieux permettre une visualisation par « tranches »

Par défaut: `0`

y-spread `float` or `int`

Permet « d'éclater » l'empilement construit vers le haut afin de mieux permettre une visualisation par « couches »

Par défaut: `0`

z-spread `float` or `int`

Permet « d'éclater » l'empilement construit vers l'avant afin de mieux permettre une visualisation par « tranches »

Par défaut: `0`

colors `array`

Pour donner une liste de 3 couleurs pour les trois faces (gauche, droite, haut)

Par défaut: `()`

grids `boolean`

Pour afficher trois grids permettant à l'élève de dessiner directement les vues de face, de dessus et de gauche

Par défaut: `false`

solution `boolean`

Affiche, lorsqu'elle est positionnée à true, les vues de face, de dessus et de gauche du solide associé sur les grids

Par défaut: `false`

below-separation `auto` or `int` or `float`

Écart de la grille du dessous (auto = calculé automatiquement)

Par défaut: `auto`

left-separation `auto` or `int` or `float`

Écart de la grille de gauche (auto = calculé automatiquement)

Par défaut: `auto`

back-separation `auto` or `int` or `float`

Écart de la grille du fond (auto = calculé automatiquement)

Par défaut: `auto`

iso `boolean` or `int`

Pour avoir différentes perspectives :
- si true, représentation isométrique,
- si 1 vue tikz3d-fr,
- si 2 première vue de ProfCollege,
- si 3 seconde vue de Profcollege,
- sinon vue par defaut de cetz.

Par défaut: `false`

space `length` or `relative length` or `fraction`

Espacement avec la seconde vue

Par défaut: `2em`

holes `boolean`

Autorise des « trous » (hauteur nulle) dans l'empilement aléatoire à partir de la deuxième pile ; la première pile reste toujours « pleine » (hauteur >= 1 partout)

Par défaut: `false`

random-mode `auto` or `int`

Mode aléatoire spécifique :
- 1 (LCG double min par gemini),
- 2 et 4 (LCG décroissant inspiré de ProfCollege par Claude),
- 3 (package suiji)

Par défaut: `auto`

views `array`

Liste des vues à afficher parmi (« dessus », « face », « droite »)

Par défaut: `("dessus", "face", "droite")`

views-names `array`

Noms des vues affichées parmi (« dessus », « face », « droite »)

Par défaut: `("Vue de dessus", "Vue de face", "Vue de droite")`

axes `boolean`

Affiche les repères d'axes (coordonnées) au sol

Par défaut: `false`

opacity `ratio`

Opacité des faces des cubes (100% = opaque, 50% = semi-transparent)

Par défaut: `100%`

shadow `boolean`

Dessine l'empreinte au sol sous l'empilement

Par défaut: `false`

disposition `str`

Disposition des grids 2D : « 3d » (projetées autour) ou « cote-a-cote » (à côté de la 3D)

Par défaut: `"3d"`

box-args `dictionary`

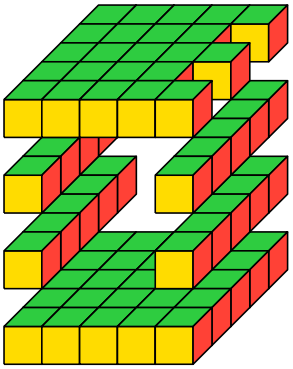
Arguments à passer à la (aux) boite(s)

Par défaut: `()`

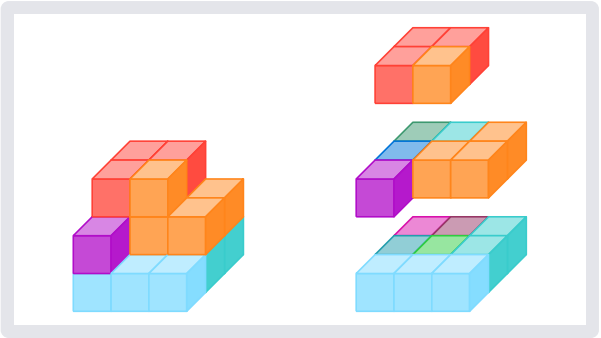
building

Assemblage de petits cubes avec mix de couleurs : chaque n° correspond à la position dans la liste de couleurs (16 max : 1 à 9 et a à g + t -> 3 couleurs), le « 0 » et le « o » représentent un trou, le « x » donne la première couleur, les cubes sont donnés dans une liste de listes : de l'avant vers l'arrière par un nombre/string « 5432a » puis étage par étage `btt`, listes de gauches à droite

```
#building((
("ttttt","ttttt","ttttt","ttttt"),
// Tranche la plus à gauche de bas en
haut
("ttttt","ooooo","ooooo","ttttt"),
("ttttt","ooooo","ooooo","ttttt"),
("ttttt","ooooo","ooooo","ttttt"),
("ttttt","ttttt","ttttt","totot")
// Tranche la plus à droite de même
), y-spread:1)
```



```
#building(((578,"21b",44), ("59a",63,64),
(533,66,0),))
#h(3em)
#building(((578,"21b",44), ("59a",63,64),
(533,66,0),),
y-spread:1.5)
```



`z:0` pour vue de face `y:0, z:(0,-1)` pour vue du dessus `x:0, z:(-1,0)` pour vue de droite

Paramètres

```
building(
  ..a: array arguments,
  color-list: array,
  x: array,
  y: array,
  z: array,
  thickness: int float ratio,
  border-stroke: boolean color,
  _rev,
  _dessous,
  _back,
  light: float ratio array,
  draw-scale: float ratio,
  iso: boolean int,
  text: str content,
  x-spread: float int,
  y-spread: float int,
  z-spread: float int,
  box-args: dictionary
) -> content
```

..a `array` or `arguments`

Liste de nombres/string correspondants aux couleurs de l'arrière vers l'avant, de bas en haut, de gauche à droite

color-list `array`

Liste de max 16 couleurs

Par défaut: `typst16`

x `array`

Direction de la première coordonnée, défaut (1,0)

Par défaut: `(1,0)`

y `array`

Direction de la seconde coordonnée, défaut (0,1)

Par défaut: `(0,1)`

z `array`

Direction de la troisième coordonnée, défaut (-0.5,-0.5)

Par défaut: `(-.5,-.5)`

thickness `int` or `float` or `ratio`

Epaisseur des tracés (de 1pt), défaut 60% soit 0.6pt

Par défaut: `60%`

border-stroke `boolean` or `color`

Lignes en noir, défaut false

Par défaut: `false`

light `float` or `ratio` or `array`

De combien éclaircir quand une seule couleur

Par défaut: `20%`

draw-scale `float` or `ratio`

Echelle du dessin : modifie la taille des arêtes d'un cube de base

Par défaut: `.5`

iso `boolean` or `int`

Pour avoir différentes perspectives :
- si true, représentation isométrique,
- si 1 vue tikz3d-fr,
- si 2 première vue de ProfCollege,
- si 3 seconde vue de Profcollege,
- sinon vue par defaut de cetz

Par défaut: `false`

text `str` or `content`

Texte à mettre en dessous de l'empilement (pour les vues)

Par défaut: none

x-spread `float` or `int`

Permet « d'éclater » l'empilement construit vers la droite afin de mieux permettre une visualisation par « tranches »

Par défaut: `0`

y-spread `float` or `int`

Permet « d'éclater » l'empilement construit vers le haut afin de mieux permettre une visualisation par « couches »

Par défaut: `0`

z-spread `float` or `int`

Permet « d'éclater » l'empilement construit vers l'avant afin de mieux permettre une visualisation par « tranches »

Par défaut: `0`

box-args `dictionary`

Arguments à passer à la boite

Par défaut: `()`

above-view
Vue du dessus pour une building

Paramètres

```
above-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: `"Vue du dessus"`

right-view
Vue de droite pour une building

Paramètres

```
right-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: `"Vue de droite"`

front-view
Vue de face pour une building

Paramètres

```
front-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: `"Vue de face"`

below-view
Vue du dessous pour une building

Paramètres

```
below-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: `"Vue du dessous"`

left-view
Vue de gauche pour une building

Paramètres

```
left-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

Nom de la vue

Par défaut: `"Vue de gauche"`

back-view
Vue de l'arrière pour une building

Paramètres

```
back-view(
  ..it: arguments,
  text: str content
) -> content
```

..it `arguments`

Arguments pour building

text `str` or `content`

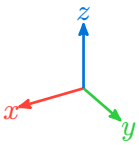
Nom de la vue

Par défaut: `"Vue arrière"`

maincoords

Fonction qui détermine les définitions pour cetz.canvas de x, y et z pour un angle de vue défini par l'élévation (θ) et l'azimut (φ). Il faut la passer directement au canvas, sans oublier les deux points

```
#canvas(..maincoords(theta:60, phi:150), {
   import draw:*
   repere
 })
```



Paramètres

```
maincoords(
  theta: int angle,
  phi: int angle
) -> dictionary
```

theta `int` or `angle`

Elévation θ

Par défaut: `70`

phi `int` or `angle`

Azimut φ

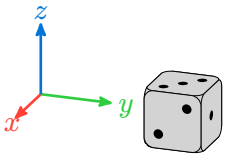
Par défaut: `110`

dice

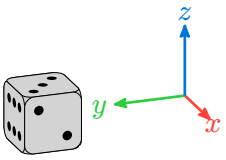
Fonction utilisée pour dessiner le dé en 3D.

Attention, l'utilisation de dice dans un canvas modifie la matrice !

```
#canvas({
   import draw:*
   dice(standalone:false, origin: (0,2),
front:2, top:3, s:.8, vue: "D")
   repere
 })
```




```
#canvas({
   import draw:*
   dice(standalone: false, origin: (0,2),
front: 2, top:3, s:.8, vue: "G")
   repere
 })
```



Paramètres

```
dice(
  standalone: boolean,
  front: int,
  top: int,
  s: int float ratio,
  theta: int float,
  phi: int float,
  vue: str,
  color: color,
  dot-color: color,
  origin: array,
  box-args: dictionary,
  opacity: ratio
) -> content function
```

standalone `boolean`

true si seul, false si déjà dans un canvas

Par défaut: `true`

front `int`

Nombre de points sur la face de devant

Par défaut: `1`

top `int`

Nombre de points sur la face du dessus

Par défaut: `2`

s `int` or `float` or `ratio`

Echelle du dessin

Par défaut: `1.0`

theta `int` or `float`

Angle theta des coordonnées sphériques

Par défaut: `70`

phi `int` or `float`

Angle phi des coordonnées sphériques

Par défaut: `110`

vue `str`

Vue « D » pour doite ou « G » pour gauche

Par défaut: `"D"`

color `color`

Couleur du dé, par défaut légèrement gris

Par défaut: `rgb("d3d3d3")`

dot-color `color`

Couleur des points

Par défaut: `black`

origin `array`

Position de l'origine du dé

Par défaut: `(0,0,0)`

box-args `dictionary`

Arguments supplémentaires pour la boîte

Par défaut: `()`

opacity `ratio`

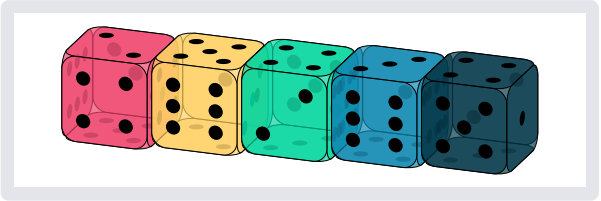
Opacité des dés

Par défaut: `100%`

throws

Tirage aléatoire ou non de n dés.

```
#throws(5, yams: true, colors:
(rgb("ef476f"), rgb("ffd166"),
rgb("06d6a0"), rgb("118ab2"),
rgb("073b4c")), seed: 999, s: 1.2,  yams-
h: -.15,  opacity: 70%,)
```



Paramètres

```
throws(
  n: int,
  list: none array,
  seed: int,
  yams: boolean,
  yams-h: int float,
  espace-h: length,
  s: int float ratio,
  colors: array,
  dot-color: color,
  theta: int float,
  phi: int float,
  vue: str,
  box-args: dictionary,
  opacity: ratio
) -> content
```

n `int`

Nombre de dés

list `none` or `array`

Liste des jets none par défaut sinon liste des couples (face avant, face du dessus),

Par défaut: `none`

seed `int`

Graine du pseudo aléatoire

Par défaut: `42`

yams `boolean`

Dés présentés « collés »

Par défaut: `false`

yams-h `int` or `float`

Avancement des dés dans la ligne si yams

Par défaut: `.1`

espace-h `length`

Espacement horizontal des dés

Par défaut: `5mm`

s `int` or `float` or `ratio`

Echelle

Par défaut: `1.0`

colors `array`

Liste des couleurs des dés

Par défaut: `(rgb("d3d3d3"),)`

dot-color `color`

couleur des points

Par défaut: `black`

theta `int` or `float`

Angle theta des coordonnées sphériques

Par défaut: `70`

phi `int` or `float`

Angle phi des coordonnées sphériques

Par défaut: `110`

vue `str`

Vue « D » droite ou « G » gauche

Par défaut: `"D"`

box-args `dictionary`

Arguments supplémentaires pour la boîte

Par défaut: `()`

opacity `ratio`

Opacité des dés

Par défaut: `100%`

dice-face

Fonction créée par eric1102 sur Discord : dessine une face d'un dé avec la possibilité de changer la couleur du dé, des points et du trait

Paramètres

```
dice-face(
  num: int,
  fill: none color,
  spot-fill: auto color,
  stroke: auto stroke
) -> content
```

num `int`

Entier de 1 à 6

fill `none` or `color`

Couleur du dé

Par défaut: `none`

spot-fill `auto` or `color`

Noir pour une couleur claire ou blanc pour une couleur sombre ou la couleur choisie

Par défaut: `auto`

stroke `auto` or `stroke`

Couleur du contour, si le dé est en couleur et stroke:auto alors même couleur

Par défaut: `auto`

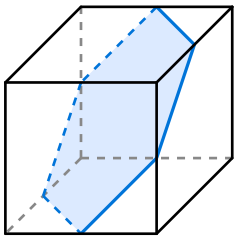
Fonctions v2 pour des solides 3D (et leurs sections et patrons)

- polysection()
- point-ratio()
- cylsection()
- conesection()
- doubleconesection()
- spheresection()
- globe-view()
- globe()
- polyunfold()
- cylunfold()
- coneunfold()
- regpolygon()
- regpyramid()
- regprism()
- regantiprism()
- regbipyramid()
- regpolyhedron()

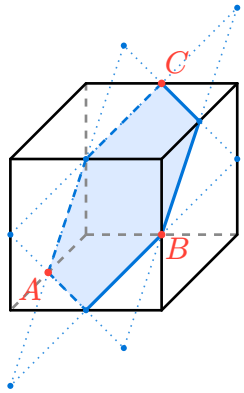
polysection

Dessine un polyèdre convexe et sa section par un plan, en projection oblique (CeTZ). Le polyèdre est défini par la liste de ses sommets. Le plan se donne par trois points ou par les quatre coefficients de son équation cartésienne. Ancres : « 0 », « 1 », … pour les sommets (dans l'ordre de `vertices`), puis les points d'intersection des prolongements (voir `show-extensions` et `show-all`) ; `help: true` les numérote.

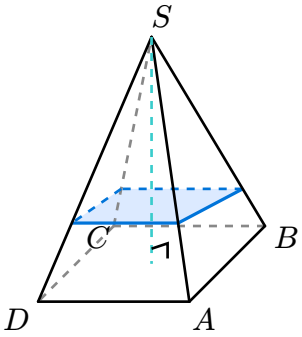
```
// Exemples d'utilisation :
#polysection((-1, -1, 0), (1, 0, 1), (0,
1, -1))
```



```
// Plan donné par trois points nommés,
avec les prolongements des côtés de la
section
#polysection((-1, -1, 0), (1, 0, 1), (0,
1, -1), plane-names: ($A$, $B$, $C$),
show-extensions: true)
```




```
// Pyramide régulière (sommets calculés
par `regpyramid`) avec le nom des sommets
#polysection(0, 1, -0.2, -0.75, vertices:
regpyramid(4, side: 2, height: 3),
  vertices-names: ($A$, $B$, $C$, $D$,
$S$), more:({draw.line("4",
("0",50%,"2"),stroke:(paint:teal,
dash:"dashed")); cetz.angle.right-
angle(("0",50%,"2"),"1","4",
radius:.2,label:none)}))
```



Paramètres

```
polysection(
  vertices: array,
  ..plane-args: array dictionary,
  vertices-names: none array,
  plane-names: none array,
  show-extensions: bool,
  show-all: bool,
  x: array,
  y: array,
  z: array,
  eps: float,
  more: cetz commands,
  section-border: color,
  section-fill: color,
  help: bool,
  hidden-style: str array dictionary,
  extensions-style: str array dictionary
) -> content
```

vertices `array`

Sommets du polyèdre convexe, sous forme de points (x, y, z) ; par défaut le cube $[-1, 1]^3$. Les fonctions de solidvertices (regpyramid, regprism, regpolyhedron…) en fournissent

Par défaut:
```
(
    (-1, -1, -1), (1, -1, -1), (1, 1, -1), (-1, 1, -1),
    (-1, -1,  1), (1, -1,  1), (1, 1,  1), (-1, 1,  1)
)
```

..plane-args `array` or `dictionary`

Plan de coupe : trois points (A, B, C), ou quatre coefficients a, b, c, d de l'équation ax + by + cz + d = 0, ou un seul argument (dictionnaire (a:, b:, c:, d:), tableau de 4 nombres ou tableau de 3 points)

vertices-names `none` or `array`

Noms des sommets, dans l'ordre de `vertices`

Par défaut: `none`

plane-names `none` or `array`

Noms des trois points qui définissent le plan, affichés en rouge (sans effet si le plan est donné par une équation)

Par défaut: `none`

show-extensions `bool`

Calcule et affiche les prolongements des côtés de la section et leurs points d'intersection

Par défaut: `false`

show-all `bool`

Affiche aussi les intersections des droites portant les côtés de la section avec les droites portant les arêtes du solide

Par défaut: `false`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (projection oblique)

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (projection oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur : par défaut (-0.5, −0.5), l'axe z pointe vers l'observateur

Par défaut: `(-0.5, -0.5)`

eps `float`

Tolérance numérique (distances)

Par défaut: `1e-6`

more `cetz commands`

Commandes CeTZ supplémentaires dessinées dans le même canevas (elles peuvent utiliser les ancres)

Par défaut: `()`

section-border `color`

Couleur du contour de la section

Par défaut: `blue`

section-fill `color`

Remplissage de la section (en général semi-transparent)

Par défaut: `rgb(0, 102, 255, 35)`

help `bool`

Affiche en rouge le numéro des ancres (sommets, puis points d'intersection)

Par défaut: `false`

hidden-style `str` or `array` or `dictionary`

Style des traits cachés : valeur de `dash` d'un trait Typst (« dashed », « dotted », …)

Par défaut: `"dashed"`

extensions-style `str` or `array` or `dictionary`

Style des lignes de rappel des prolongements

Par défaut: `"dotted"`

point-ratio

Point situé à une fraction du segment [A C], en partant de A (en 2D ou en 3D). `ratio` est un nombre entre 0 et 1 ou un pourcentage.

```
#point-ratio((0, 2, 2), 25%, (4, 0, 10))
```

```
(1.0, 1.5, 4.0)
```

Paramètres

```
point-ratio(
  A: array,
  ratio: int float ratio,
  C: array
) -> array
```

A `array`

Point de départ (x, y) ou (x, y, z)

ratio `int` or `float` or `ratio`

Fraction du segment [A C] : nombre (0.25) ou pourcentage (25%)

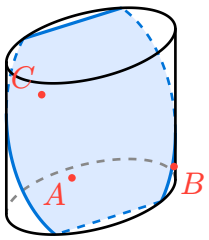
C `array`

Point d'arrivée, de même dimension que A

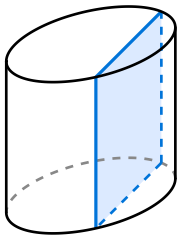
cylsection

Dessine un cylindre et sa section par un plan, en projection oblique (CeTZ). Le cylindre est défini par son rayon, sa hauteur, le centre de sa base et la direction de son axe. Le plan se donne comme pour `polysection` (trois points ou quatre coefficients). Les cas particuliers sont traités : plan parallèle à l'axe (rectangle), perpendiculaire à l'axe (cercle) ; l'ellipse peut montrer ses foyers. Ancres : « c0 », « c1 » (centres des bases), « i0 », « i1 », … (extrémités des cordes de la section sur les bases), « sc » (centre de la section quand c'est une ellipse complète), « f0 », « f1 » (foyers). Avec `show-dims`, la section montre ses dimensions : rayon d'un cercle, demi-axes a et b d'une ellipse complète (et c, distance du centre à un foyer, avec `show-foci`). Les valeurs `auto` de `radius-label` et de `dims-labels` affichent les vraies valeurs (arrondies, précédées de ≈ si besoin ; `label-unit`, `label-digits`, `label-decimal`).

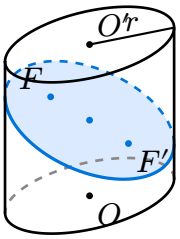
```
// Exemples d'utilisation :
#cylsection((0, 0.5, 0.5), (1, 0.3, -0.2),
(-0.6, 1.4, 0.1), plane-names: ($A$, $B$,
$C$))
```



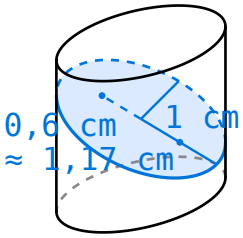

```
  // Plan parallèle à l'axe (rectangle)
  #cylsection(1, 0, 0, -0.5, radius: 1,
  height: 2)
```



```
// Ellipse complète avec un rayon, les
centres et les foyers
#cylsection(0.6, 1, 0, -1, show-radius:
true, show-centers: true, show-foci: true,
  point-labels: (c0: $O$, c1: $O'$, f0:
$F$, f1: $F'$))
```




```
// Dimensions de l'ellipse (a, b, c) avec
leurs vraies valeurs
#cylsection(0.6, 1, 0, -1, show-dims:
true, show-foci: true, dims-labels: auto,
label-unit: "cm")
```



Paramètres

```
cylsection(
  ..plane-args: array dictionary,
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  plane-names: none array,
  show-axis: bool,
  show-foci: bool,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Plan de coupe : trois points (A, B, C), ou quatre coefficients a, b, c, d de l'équation ax + by + cz + d = 0, ou un seul argument (dictionnaire (a:, b:, c:, d:), tableau de 4 nombres ou tableau de 3 points)

radius `int` or `float`

Rayon du cylindre

Par défaut: 1

height `int` or `float`

Hauteur du cylindre (distance entre les deux bases)

Par défaut: 2

base `array`

Centre de la base « inférieure »

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, de la base vers la face supérieure (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

plane-names `none` or `array`

Noms des trois points qui définissent le plan, affichés en rouge (sans effet si le plan est donné par une équation)

Par défaut: `none`

show-axis `bool`

Trace l'axe en tirets-points

Par défaut: `false`

show-foci `bool`

Marque les foyers de la section quand ils existent (ellipse, parabole, hyperbole)

Par défaut: `false`

show-radius `bool`

Trace un rayon, avec son étiquette, sur la base visible

Par défaut: `false`

radius-label `auto` or `none` or `content`

Étiquette du rayon ; auto : sa valeur numérique ; none pour ne pas en mettre

Par défaut: `$r$`

radius-stroke `stroke`

Trait du rayon

Par défaut: `black + 0.8pt`

show-centers `bool`

Marque les centres des bases et le centre de la section quand il est évident

Par défaut: `false`

show-dims `bool`

Trace les dimensions de la section avec leurs étiquettes : rayon d'un cercle, ou demi-axes a et b d'une ellipse complète (et c, distance du centre à un foyer, avec show-foci)

Par défaut: `false`

dims-labels `auto` or `dictionary`

Étiquettes des dimensions de la section : dictionnaire (r:, a:, b:, c:) dont les valeurs remplacent les étiquettes symboliques (r', a, b, c) (la valeur auto donne la vraie valeur), ou auto pour tout remplacer

Par défaut: `(:)`

point-labels `dictionary`

Étiquettes posées sur des ancres (clé = nom de l'ancre, valeur = contenu), par exemple (c0: O, s: S) ; elles se placent vers l'extérieur du solide

Par défaut: `(:)`

label-unit `none` or `str`

Unité ajoutée aux longueurs en mode auto, par exemple « cm »

Par défaut: `none`

label-digits `int`

Nombre de décimales en mode auto

Par défaut: `2`

label-decimal `str`

Séparateur décimal en mode auto : « , » ou « . »

Par défaut: `","`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (projection oblique)

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (projection oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur : par défaut (-0.5, −0.5), l'axe z pointe vers l'observateur

Par défaut: `(-0.5, -0.5)`

eps `float`

Tolérance numérique (distances)

Par défaut: `1e-6`

per-turn `int`

Nombre de segments utilisés pour tracer un tour complet de cercle ou d'ellipse

Par défaut: `144`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas (elles peuvent utiliser les ancres)

Par défaut: `()`

section-border `color`

Couleur du contour de la section

Par défaut: `blue`

section-fill `color`

Remplissage de la section (en général semi-transparent)

Par défaut: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Style des traits cachés : valeur de `dash` d'un trait Typst (« dashed », « dotted », …)

Par défaut: `"dashed"`

length `length`

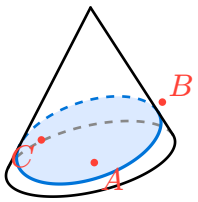
Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

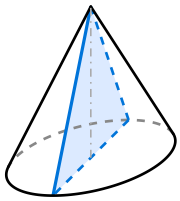
conesection

Dessine un cône et sa section par un plan, en projection oblique (CeTZ). Le cône est défini par le rayon et le centre de sa base, sa hauteur et la direction de son axe. Le plan se donne comme pour `polysection`. Selon le plan, la section est un cercle, une ellipse, une parabole ou un morceau d'hyperbole ; un plan passant par le sommet donne un triangle. Les foyers sont calculés avec les sphères de Dandelin. Ancres : « c0 » (centre de la base), « s » (sommet), « i0 », « i1 », … (extrémités de la corde sur la base), « sc » (centre de la section quand c'est un cercle), « f0 », « f1 » (foyers). Avec `show-dims`, la section montre ses dimensions : rayon d'un cercle, demi-axes a et b d'une ellipse complète (et c, distance du centre à un foyer, avec `show-foci`). Les valeurs `auto` de `radius-label` et de `dims-labels` affichent les vraies valeurs (arrondies, précédées de ≈ si besoin ; `label-unit`, `label-digits`, `label-decimal`).

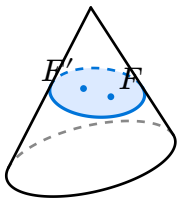
```
// Exemples d'utilisation :
#conesection((0.3, 0.2, 0.5), (0.6, 0.4,
-0.7), (-0.6, 0.3, 0.1), plane-names:
($A$, $B$, $C$))
```



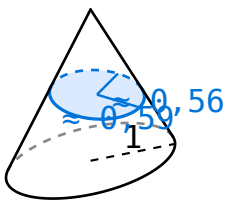

```
// Plan contenant l'axe (triangle)
#conesection(1, 0, 0, 0, show-axis: true)
```



```
// Ellipse avec ses foyers
#conesection(0.3, 1, 0, -0.9, show-foci:
true, point-labels: (f0: $F$, f1: $F'$))
```




```
// Dimensions de l'ellipse et rayon de la
base avec leurs vraies valeurs
#conesection(0.3, 1, 0, -0.9, show-dims:
true, dims-labels: auto, show-radius:
true, radius-label: auto)
```



Paramètres

```
conesection(
  ..plane-args: array dictionary,
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  plane-names: none array,
  show-axis: bool,
  show-foci: bool,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-apex: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Plan de coupe : trois points (A, B, C), ou quatre coefficients a, b, c, d de l'équation ax + by + cz + d = 0, ou un seul argument (dictionnaire (a:, b:, c:, d:), tableau de 4 nombres ou tableau de 3 points)

radius `int` or `float`

Rayon de la base

Par défaut: 1

height `int` or `float`

Hauteur du cône (distance de la base au sommet)

Par défaut: 2

base `array`

Centre de la base

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, de la base vers le sommet (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

plane-names `none` or `array`

Noms des trois points qui définissent le plan, affichés en rouge (sans effet si le plan est donné par une équation)

Par défaut: `none`

show-axis `bool`

Trace l'axe en tirets-points

Par défaut: `false`

show-foci `bool`

Marque les foyers de la section quand ils existent (ellipse, parabole, hyperbole)

Par défaut: `false`

show-radius `bool`

Trace un rayon, avec son étiquette, sur la base (en pointillés si elle est cachée)

Par défaut: `false`

radius-label `auto` or `none` or `content`

Étiquette du rayon ; auto : sa valeur numérique ; none pour ne pas en mettre

Par défaut: `$r$`

radius-stroke `stroke`

Trait du rayon

Par défaut: `black + 0.8pt`

show-centers `bool`

Marque le centre de la base et le centre de la section quand c'est un cercle

Par défaut: `false`

show-apex `bool`

Marque le sommet

Par défaut: `false`

show-dims `bool`

Trace les dimensions de la section avec leurs étiquettes : rayon d'un cercle, ou demi-axes a et b d'une ellipse complète (et c, distance du centre à un foyer, avec show-foci)

Par défaut: `false`

dims-labels `auto` or `dictionary`

Étiquettes des dimensions de la section : dictionnaire (r:, a:, b:, c:) dont les valeurs remplacent les étiquettes symboliques (r', a, b, c) (la valeur auto donne la vraie valeur), ou auto pour tout remplacer

Par défaut: `(:)`

point-labels `dictionary`

Étiquettes posées sur des ancres (clé = nom de l'ancre, valeur = contenu), par exemple (c0: O, s: S) ; elles se placent vers l'extérieur du solide

Par défaut: `(:)`

label-unit `none` or `str`

Unité ajoutée aux longueurs en mode auto, par exemple « cm »

Par défaut: `none`

label-digits `int`

Nombre de décimales en mode auto

Par défaut: `2`

label-decimal `str`

Séparateur décimal en mode auto : « , » ou « . »

Par défaut: `","`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (projection oblique)

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (projection oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur : par défaut (-0.5, −0.5), l'axe z pointe vers l'observateur

Par défaut: `(-0.5, -0.5)`

eps `float`

Tolérance numérique (distances)

Par défaut: `1e-6`

per-turn `int`

Nombre de segments utilisés pour tracer un tour complet de cercle ou d'ellipse

Par défaut: `144`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas (elles peuvent utiliser les ancres)

Par défaut: `()`

section-border `color`

Couleur du contour de la section

Par défaut: `blue`

section-fill `color`

Remplissage de la section (en général semi-transparent)

Par défaut: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Style des traits cachés : valeur de `dash` d'un trait Typst (« dashed », « dotted », …)

Par défaut: `"dashed"`

length `length`

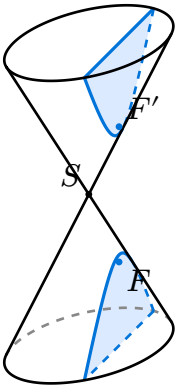
Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

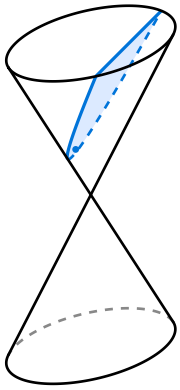
doubleconesection

Dessine un double cône (« sablier ») et sa section par un plan, en projection oblique (CeTZ). Deux cônes de même sommet et de même axe, opposés par le sommet ; chacun a pour hauteur `height` et pour rayon de base `radius`. Les sections possibles : cercle, ellipse, parabole, hyperbole (deux branches, une par nappe), deux droites sécantes (plan passant par le sommet), point ou droite. Ancres : « s » (sommet), « c0 » (centre de la base du bas), « c1 » (centre de la base du haut), « i0 », « i1 », … (extrémités des cordes sur les bases), « sc » (centre d'une section circulaire), « f0 », « f1 » (foyers). Limite : le tracé suppose que les deux nappes ne se masquent pas l'une l'autre, ce qui est le cas tant que l'angle entre la direction de vue et l'axe dépasse le demi-angle au sommet. Avec `show-dims`, la section montre ses dimensions : rayon d'un cercle, demi-axes a et b d'une ellipse complète (et c, distance du centre à un foyer, avec `show-foci`). Les valeurs `auto` de `radius-label` et de `dims-labels` affichent les vraies valeurs (arrondies, précédées de ≈ si besoin ; `label-unit`, `label-digits`, `label-decimal`).

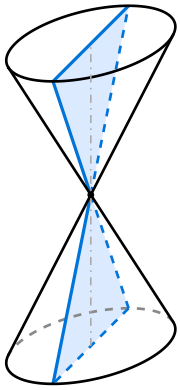
```
// Exemples d'utilisation :
#doubleconesection(1, 0, 0, -0.4, show-
foci: true, show-apex: true,
  point-labels: (s: $S$, f0: $F$, f1:
$F'$))
```



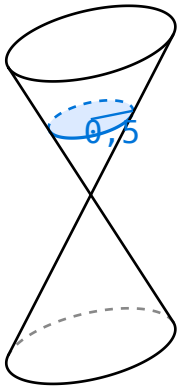
```
// Parabole (plan parallèle à une
génératrice)
#doubleconesection(0.894, -0.447, 0,
0.447, show-foci: true)
```



```
// Plan passant par le sommet
#doubleconesection(1, 0, 0, 0, show-axis:
true, show-apex: true)
```



```
// Rayon du cercle de section (plan
perpendiculaire à l'axe) avec sa vraie
valeur
#doubleconesection(0, 1, 0, -1, show-dims:
true, dims-labels: auto)
```



Paramètres

```
doubleconesection(
  ..plane-args: array dictionary,
  radius: int float,
  height: int float,
  apex: array,
  axis: array,
  plane-names: none array,
  show-axis: bool,
  show-foci: bool,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-apex: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Plan de coupe : trois points (A, B, C), ou quatre coefficients a, b, c, d de l'équation ax + by + cz + d = 0, ou un seul argument (dictionnaire (a:, b:, c:, d:), tableau de 4 nombres ou tableau de 3 points)

radius `int` or `float`

Rayon de la base de chaque nappe

Par défaut: 1

height `int` or `float`

Hauteur de chaque nappe (distance du sommet à chaque base)

Par défaut: 2

apex `array`

Sommet commun des deux cônes

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, de la nappe du bas vers la nappe du haut (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

plane-names `none` or `array`

Noms des trois points qui définissent le plan, affichés en rouge (sans effet si le plan est donné par une équation)

Par défaut: `none`

show-axis `bool`

Trace l'axe en tirets-points

Par défaut: `false`

show-foci `bool`

Marque les foyers de la section quand ils existent (ellipse, parabole, hyperbole)

Par défaut: `false`

show-radius `bool`

Trace un rayon, avec son étiquette, sur la base tournée vers l'observateur

Par défaut: `false`

radius-label `auto` or `none` or `content`

Étiquette du rayon ; auto : sa valeur numérique ; none pour ne pas en mettre

Par défaut: `$r$`

radius-stroke `stroke`

Trait du rayon

Par défaut: `black + 0.8pt`

show-centers `bool`

Marque les centres des deux bases et le centre de la section quand c'est un cercle

Par défaut: `false`

show-apex `bool`

Marque le sommet

Par défaut: `false`

show-dims `bool`

Trace les dimensions de la section avec leurs étiquettes : rayon d'un cercle, ou demi-axes a et b d'une ellipse complète (et c, distance du centre à un foyer, avec show-foci)

Par défaut: `false`

dims-labels `auto` or `dictionary`

Étiquettes des dimensions de la section : dictionnaire (r:, a:, b:, c:) dont les valeurs remplacent les étiquettes symboliques (r', a, b, c) (la valeur auto donne la vraie valeur), ou auto pour tout remplacer

Par défaut: `(:)`

point-labels `dictionary`

Étiquettes posées sur des ancres (clé = nom de l'ancre, valeur = contenu), par exemple (c0: O, s: S) ; elles se placent vers l'extérieur du solide

Par défaut: `(:)`

label-unit `none` or `str`

Unité ajoutée aux longueurs en mode auto, par exemple « cm »

Par défaut: `none`

label-digits `int`

Nombre de décimales en mode auto

Par défaut: `2`

label-decimal `str`

Séparateur décimal en mode auto : « , » ou « . »

Par défaut: `","`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (projection oblique)

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (projection oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur : par défaut (-0.5, −0.5), l'axe z pointe vers l'observateur

Par défaut: `(-0.5, -0.5)`

eps `float`

Tolérance numérique (distances)

Par défaut: `1e-6`

per-turn `int`

Nombre de segments utilisés pour tracer un tour complet de cercle ou d'ellipse

Par défaut: `144`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas (elles peuvent utiliser les ancres)

Par défaut: `()`

section-border `color`

Couleur du contour de la section

Par défaut: `blue`

section-fill `color`

Remplissage de la section (en général semi-transparent)

Par défaut: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Style des traits cachés : valeur de `dash` d'un trait Typst (« dashed », « dotted », ...)

Par défaut: `"dashed"`

length `length`

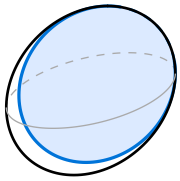
Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

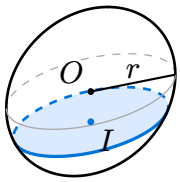
spheresection

Dessine une sphère et sa section par un plan, en projection oblique (CeTZ). Le contour apparent est calculé exactement (c'est une ellipse en projection oblique) ; un équateur facultatif donne l'impression de relief. Ancres : « o » (centre), « sc » (centre du cercle de section). Avec `show-dims`, le rayon du cercle de section est tracé et étiqueté. Les valeurs `auto` de `radius-label` et de `dims-labels` affichent les vraies valeurs (arrondies, précédées de ≈ si besoin ; `label-unit`, `label-digits`, `label-decimal`).

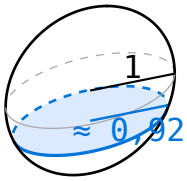
```
// Exemples d'utilisation :
#spheresection(1, 1, 1, -0.5)
```



```
// Plan horizontal avec un rayon, les centres et des étiquettes
#spheresection(0, 1, 0, -0.6, center: (0, 1, 0), show-radius: true, show-centers: true,
  point-labels: (o: $O$, sc: $I$))
```




```
// Rayon de la sphère et du cercle de section avec leurs vraies valeurs
#spheresection(0, 1, 0, -0.6, center: (0, 1, 0), show-radius: true, radius-label: auto, show-dims: true, dims-labels: auto)
```



Paramètres

```
spheresection(
  ..plane-args: array dictionary,
  radius: int float,
  center: array,
  axis: array,
  show-equator: bool,
  plane-names: none array,
  show-radius: bool,
  radius-label: auto none content,
  radius-stroke: stroke,
  show-centers: bool,
  show-dims: bool,
  dims-labels: auto dictionary,
  point-labels: dictionary,
  label-unit: none str,
  label-digits: int,
  label-decimal: str,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Plan de coupe : trois points (A, B, C), ou quatre coefficients a, b, c, d de l'équation ax + by + cz + d = 0, ou un seul argument (dictionnaire (a:, b:, c:, d:), tableau de 4 nombres ou tableau de 3 points)

radius `int` or `float`

Rayon de la sphère

Par défaut: `1`

center `array`

Centre de la sphère

Par défaut: `(0, 0, 0)`

axis `array`

Direction des pôles (sert seulement à tracer l'équateur)

Par défaut: `(0, 1, 0)`

show-equator `bool`

Trace l'équateur (cercle perpendiculaire à l'axe), qui donne du relief

Par défaut: `true`

plane-names `none` or `array`

Noms des trois points qui définissent le plan, affichés en rouge (sans effet si le plan est donné par une équation)

Par défaut: `none`

show-radius `bool`

Trace un rayon sur le contour apparent, où il apparaît en vraie grandeur

Par défaut: `false`

radius-label `auto` or `none` or `content`

Étiquette du rayon ; auto : sa valeur numérique ; none pour ne pas en mettre

Par défaut: `$r$`

radius-stroke `stroke`

Trait du rayon

Par défaut: `black + 0.8pt`

show-centers `bool`

Marque le centre de la sphère et celui du cercle de section

Par défaut: `false`

show-dims `bool`

Trace les dimensions de la section avec leurs étiquettes : le rayon du cercle de section

Par défaut: `false`

dims-labels `auto` or `dictionary`

Étiquettes des dimensions de la section : dictionnaire (r:, a:, b:, c:) dont les valeurs remplacent les étiquettes symboliques (r', a, b, c) (la valeur auto donne la vraie valeur), ou auto pour tout remplacer

Par défaut: `(:)`

point-labels `dictionary`

Étiquettes posées sur des ancres (clé = nom de l'ancre, valeur = contenu), par exemple (o: O, sc: I)

Par défaut: `(:)`

label-unit `none` or `str`

Unité ajoutée aux longueurs en mode auto, par exemple « cm »

Par défaut: none

label-digits `int`

Nombre de décimales en mode auto

Par défaut: 2

label-decimal `str`

Séparateur décimal en mode auto : « , » ou « . »

Par défaut: ","

x `array`

Image à l'écran du vecteur unitaire de l'axe x (projection oblique)

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (projection oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur : par défaut (-0.5, −0.5), l'axe z pointe vers l'observateur

Par défaut: `(-0.5, -0.5)`

eps `float`

Tolérance numérique (distances)

Par défaut: `1e-6`

per-turn `int`

Nombre de segments utilisés pour tracer un tour complet de cercle ou d'ellipse

Par défaut: `144`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas (elles peuvent utiliser les ancres)

Par défaut: `()`

section-border `color`

Couleur du contour de la section

Par défaut: `blue`

section-fill `color`

Remplissage de la section (en général semi-transparent)

Par défaut: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Style des traits cachés : valeur de `dash` d'un trait Typst (« dashed », « dotted », …)

Par défaut: `"dashed"`

length `length`

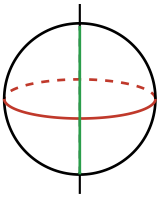
Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

globe-view
Paramètres de vue (x, y, z) d'une projection orthographique centrée sur le point de longitude `lon` et de latitude `lat`. La sphère apparaît alors comme un cercle, avec l'axe sud-nord vertical. À utiliser avec l'orientation par défaut de `globe` (axe (0, 1, 0), Greenwich (0, 0, 1)). Plus `lat` est grand, plus

l'équateur s'ouvre (effet de relief) et plus le pôle Nord s'éloigne du bord haut du dessin ; entre 10 et 20, il en paraît pratiquement le point le plus haut.

```
// Exemple d'utilisation :
#globe(..globe-view(lon: 0, lat: 15),
show-axis: true)
```



Paramètres

```
globe-view(
  lon: int float,
  lat: int float
) -> dictionary
```

lon `int` or `float`

Longitude du point au centre du dessin, en degrés (est positif)

Par défaut: `20`

lat `int` or `float`

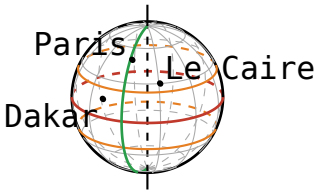
Latitude du point au centre du dessin, en degrés (nord positif)

Par défaut: `25`

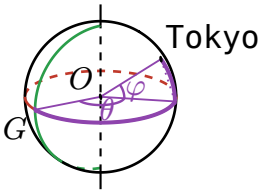
globe

Dessine un globe terrestre (variante de `spheresection`) pour expliquer les coordonnées géographiques. Latitude : angle depuis l'équateur, de −90° (sud) à +90° (nord). Longitude : angle depuis le méridien de Greenwich, de −180° (ouest) à +180° (est). Tous les angles se donnent en degrés. On peut afficher l'équateur, les tropiques, le méridien de Greenwich, des parallèles et des méridiens (isolés ou en quadrillage), l'axe sud-nord, des villes (voir `globe-cities` : 50 grandes villes) ou des points de coordonnées quelconques et, pour un point, la construction de ses coordonnées : angle de longitude dans le plan de l'équateur et angle de latitude dans le plan de son méridien. Un plan peut aussi être donné pour dessiner une section (un parallèle est un plan perpendiculaire à l'axe, un méridien un plan qui le contient). La projection oblique déforme la sphère en ellipse : `globe-view` donne une vue orthographique. Ancres : « o » (centre), « n », « s » (pôles), « g » (point de coordonnées (0, 0)), « p0 », « p1 », … et le nom de chaque point, « sc » (centre de la section). Avec `angles: auto` pour un point (ou `lon-label: auto` / `lat-label: auto` pour tous), les angles de la construction affichent leurs valeurs en degrés au lieu de θ et φ (`label-digits`, `label-decimal`).

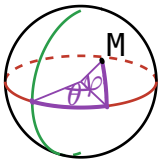
```
// Exemples d'utilisation :
#globe(..globe-view(lon: 20, lat: 20),
show-axis: true, show-tropics: true,
graticule: 30,
  points: ("Paris", "Le Caire", "Dakar"))
```



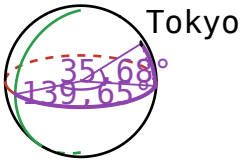

```
// Construction de la longitude et de la
latitude d'une ville
#globe(..globe-view(lon: 60, lat: 20),
show-axis: true, show-centers: true,
  points: (("city": "Tokyo", angles:
true),), point-labels: (o: $O$, g: $G$))
```




```
// Point de coordonnées quelconques
#globe(..globe-view(lon: 40, lat: 20),
points: ((name: "M", lat: 35, lon: 60,
angles: true),))
```




```
// Construction avec les valeurs
numériques de la longitude et de la
latitude
#globe(..globe-view(lon: 60, lat: 20),
points: (("city": "Tokyo", angles:
auto),))
```



Paramètres

```
globe(
  ..plane-args: array dictionary,
  radius: int float,
  center: array,
  axis: array,
  greenwich: array,
  show-equator: bool,
  show-greenwich: bool,
  show-tropics: bool,
  parallels: array,
  meridians: array,
  graticule: none int float,
  show-axis: bool,
  axis-extension: int float,
  show-centers: bool,
  points: array,
  lon-label: auto content,
  lat-label: auto content,
  label-digits: int,
  label-decimal: str,
  angle-color: color,
  equator-stroke: stroke,
  greenwich-stroke: stroke,
  tropics-stroke: stroke,
  grid-stroke: stroke,
  axis-stroke: stroke,
  plane-names: none array,
  point-labels: dictionary,
  x: array,
  y: array,
  z: array,
  eps: float,
  per-turn: int,
  more: content,
  section-border: color,
  section-fill: color,
  hidden-style: str array dictionary,
  length: length
) -> content
```

..plane-args `array` or `dictionary`

Plan de coupe : trois points (A, B, C), ou quatre coefficients a, b, c, d de l'équation ax + by + cz + d = 0, ou un seul argument (dictionnaire (a:, b:, c:, d:), tableau de 4 nombres ou tableau de 3 points)

radius `int` or `float`

Rayon de la sphère

Par défaut: 1

center `array`

Centre de la sphère

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe sud-nord, du pôle Sud vers le pôle Nord

Par défaut: `(0, 1, 0)`

greenwich `array`

Direction du point de latitude 0 et de longitude 0, projetée perpendiculairement à l'axe ; avec les valeurs par défaut, le méridien de Greenwich fait face à l'observateur et l'est est à droite

Par défaut: `(0, 0, 1)`

show-equator `bool`

Trace l'équateur

Par défaut: `true`

show-greenwich `bool`

Trace le méridien de Greenwich (demi-cercle de pôle à pôle)

Par défaut: `true`

show-tropics `bool`

Trace les tropiques du Cancer et du Capricorne (latitudes ±23,44°)

Par défaut: `false`

parallels `array`

Latitudes supplémentaires à tracer, en degrés, par exemple (30, −45)

Par défaut: `()`

meridians `array`

Longitudes supplémentaires à tracer, en degrés, par exemple (90, −120)

Par défaut: `()`

graticule `none` or `int` or `float`

Pas, en degrés, d'un quadrillage de parallèles et de méridiens (par exemple 30) ; none pour aucun

Par défaut: `none`

show-axis `bool`

Trace l'axe sud-nord (en pointillés à l'intérieur de la sphère et derrière elle)

Par défaut: `false`

axis-extension `int` or `float`

Longueur dont l'axe dépasse chaque pôle, en fraction du rayon

Par défaut: `0.3`

show-centers `bool`

Marque le centre de la sphère et celui de la section

Par défaut: `false`

points `array`

Points à placer : noms de villes de `globe-cities` ou dictionnaires (name:, lat:, lon:, color:, label:, angles:) ; `angles: true` ajoute la construction des coordonnées du point, `angles: auto` la fait avec les valeurs numériques des angles

Par défaut: `()`

lon-label `auto` or `content`

Étiquette de l'angle de longitude ; auto : sa valeur en degrés

Par défaut: `$theta$`

lat-label `auto` or `content`

Étiquette de l'angle de latitude ; auto : sa valeur en degrés

Par défaut: `$phi$`

label-digits `int`

Nombre de décimales des angles en mode auto

Par défaut: `2`

label-decimal `str`

Séparateur décimal des angles en mode auto : « , » ou « . »

Par défaut: `","`

angle-color `color`

Couleur de la construction des angles

Par défaut: `rgb("#8e44ad")`

equator-stroke `stroke`

Trait de l'équateur

Par défaut: `rgb("#c0392b") + 1pt`

greenwich-stroke `stroke`

Trait du méridien de Greenwich

Par défaut: `rgb("#2a9d4a") + 1pt`

tropics-stroke `stroke`

Trait des tropiques

Par défaut: `rgb("#e67e22") + 0.8pt`

grid-stroke `stroke`

Trait des parallèles et méridiens du quadrillage

Par défaut: `gray + 0.4pt`

axis-stroke `stroke`

Trait de l'axe sud-nord

Par défaut: `black + 0.8pt`

plane-names `none` or `array`

Noms des trois points qui définissent le plan, affichés en rouge (sans effet si le plan est donné par une équation)

Par défaut: `none`

point-labels `dictionary`

Étiquettes posées sur des ancres (clé = nom de l'ancre, valeur = contenu), par exemple (o: O, n: N)

Par défaut: `(:)`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (projection oblique)

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (projection oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur : par défaut (-0.5, −0.5), l'axe z pointe vers l'observateur

Par défaut: `(-0.5, -0.5)`

eps `float`

Tolérance numérique (distances)

Par défaut: `1e-6`

per-turn `int`

Nombre de segments utilisés pour tracer un tour complet de cercle ou d'ellipse

Par défaut: `144`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas (elles peuvent utiliser les ancres)

Par défaut: `()`

section-border `color`

Couleur du contour de la section

Par défaut: `blue`

section-fill `color`

Remplissage de la section (en général semi-transparent)

Par défaut: `rgb(0, 102, 255, 35)`

hidden-style `str` or `array` or `dictionary`

Style des traits cachés : valeur de `dash` d'un trait Typst (« dashed », « dotted », ...)

Par défaut: `"dashed"`

length `length`

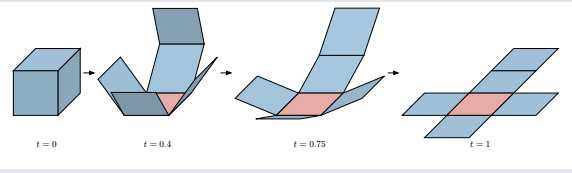
Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

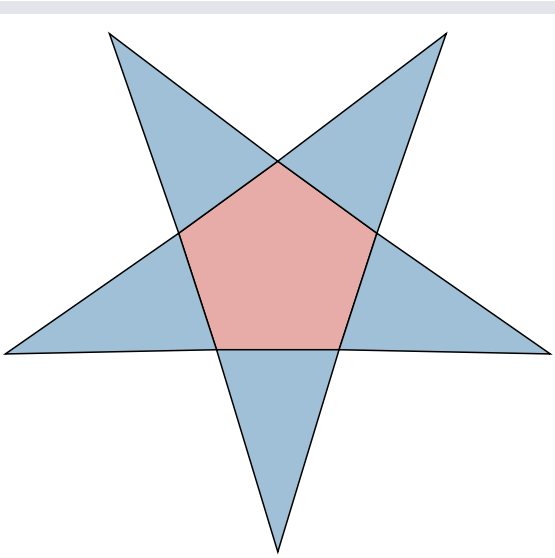
polyunfold

Affiche un polyèdre convexe, son patron et les étapes intermédiaires de son dépliage (CeTZ). Le paramètre d'avancement t va de 0 (solide fermé) à 1 (patron à plat). Les étapes demandées (`steps`) sont dessinées côte à côte, à la même échelle. Le solide est posé sur sa face du dessous, qui reste fixe : les autres faces se déplient autour d'elle. Le patron est choisi parmi plusieurs arbres de dépliage (parcours en largeur et en profondeur) : le premier dont les faces ne se recouvrent pas est retenu. Les faces sont opaques, ombrées et triées de la plus lointaine à la plus proche ; le solide fermé est dessiné sans arêtes cachées. Ancres, suffixées −1, −2, ... s'il y a plusieurs étapes : « f0 », « f1 », ... (centres des faces) et « v0 », « v1 », ... (sommets, dans l'ordre de `vertices` ; un sommet est pris sur la première face de l'arbre qui le contient).

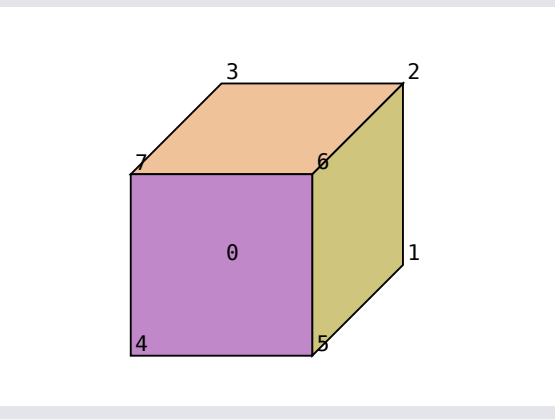
```
// Exemples d'utilisation :
#move(polyunfold(steps: (0, 0.4, 0.75, 1),
labels: ($t = 0$, $t = 0.4$, $t = 0.75$,
$t = 1$)),dy:1.85cm)
```



```
// Patron à plat, en vraie grandeur, d'une pyramide
#move(polyunfold(vertices: regpyramid(5, side: 2, height: 3), view: "top"),dy:.9cm)
```



```
// Une couleur par face et numéros des sommets
#polyunfold(t: 0, length: 1.2cm,
  palette: (red, green, blue, yellow, orange, purple).map(c => c.lighten(60%)),
  more: {
    import cetz.draw: *
    for i in range(8) {
      content("v" + str(i), text(size: 8pt, str(i)), anchor: "south-west", padding: 0.05)
    }
  })
```



Paramètres

```
polyunfold(
  vertices: array,
  steps: auto array,
  t: none int float,
  root: auto int,
  up: array,
  view: str,
  columns: auto int,
  gap: int float,
  labels: none array,
  arrows: bool,
  fill: color,
  base-fill: color,
  palette: none array,
  edge-stroke: stroke,
  x: array,
  y: array,
  z: array,
  more: content,
  length: length,
  angle: angle
) -> content
```

vertices `array`

Sommets du polyèdre convexe (par défaut le cube $[-1, 1]^3$), comme pour polysection ; les fonctions de solidvertices en fournissent

Par défaut: `(`

```
    (-1, -1, -1), (1, -1, -1), (1, 1, -1), (-1, 1, -1),
    (-1, -1, 1), (1, -1, 1), (1, 1, 1), (-1, 1, 1),
  )
```

steps `auto` or `array`

Valeurs du paramètre de dépliage t à afficher côte à côte, de 0 (solide fermé) à 1 (patron à plat). Par défaut (0, 0.5, 1), ou (1,) avec view: « top »

Par défaut: `auto`

t `none` or `int` or `float`

Une seule valeur de t (prioritaire sur steps)

Par défaut: `none`

root `auto` or `int`

Indice de la face qui reste fixe ; auto : la face du dessous

Par défaut: `auto`

up `array`

Direction « vers le haut », utilisée pour reconnaître la face du dessous

Par défaut: `(0, 1, 0)`

view `str`

« oblique » : projection oblique x, y, z (par défaut) ; « top » : patron à plat vu de dessus, en vraie grandeur, orienté comme la vue oblique

Par défaut: `"oblique"`

columns `auto` or `int`

Nombre de figures par ligne ; auto : toutes sur une seule ligne

Par défaut: `auto`

gap `int` or `float`

Espace entre les figures (unités du repère)

Par défaut: `0.8`

labels `none` or `array`

Légendes placées sous les figures (une par étape)

Par défaut: `none`

arrows `bool`

Flèches entre deux figures consécutives d'une même ligne

Par défaut: `true`

fill `color`

Couleur des faces

Par défaut: `rgb("#a9cce3")`

base-fill `color`

Couleur de la face qui reste fixe (la base)

Par défaut: `rgb("#f5b7b1")`

palette `none` or `array`

Couleurs des faces, répétées si besoin ; remplace fill et base-fill

Par défaut: `none`

edge-stroke `stroke`

Trait des arêtes

Par défaut: `0.7pt + black`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (vue oblique, comme pour polysection) ; sans effet avec view: « top »

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (vue oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur (vue oblique)

Par défaut: `(-0.5, -0.5)`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas ; la première figure garde ses coordonnées d'origine, les ancres existent pour toutes

Par défaut: `()`

length `length`

Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

angle `angle`

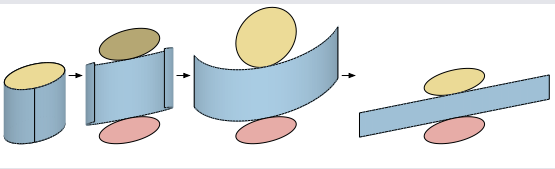
Angle de rotation du dessin en view:« top »

Par défaut: `0deg`

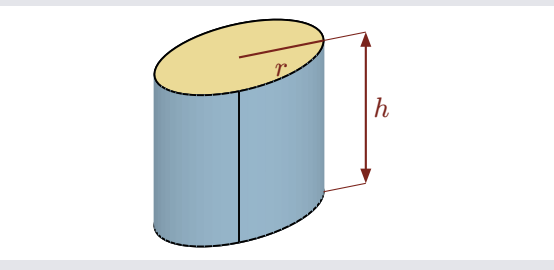
cylunfold

Affiche un cylindre, son patron et les étapes intermédiaires de son dépliage (CeTZ). Le paramètre d'avancement t va de 0 (solide fermé) à 1 (patron à plat). Le disque de base reste fixe ; la surface latérale se déroule (la longueur de ses arcs est conservée) en un rectangle de longueur 2πr tangent au disque, et le second disque bascule avec elle. Les faces sont opaques, ombrées et triées de la plus lointaine à la plus proche. Avec `cotes: true`, on ajoute les cotes (r, h sur le solide fermé ; 2πr, h, r sur le patron) ; les étiquettes sont à l'extérieur pour les hauteurs et sous les lignes pour les autres. Ancres, suffixées −1, −2, … s'il y a plusieurs étapes : « c0 » (centre du disque fixe), « c1 » (centre du second disque), « pc », « qc » (points de contact, en bas et en haut), « cote-r », « cote-h », « cote-p » (milieux des lignes de cote). Avec `base-ratio`, on choisit où chaque disque touche la surface latérale du patron (ratio mesuré depuis l'extrémité gauche en vue de dessus) : un nombre pour les deux disques, ou (bas, haut) pour les placer indépendamment.

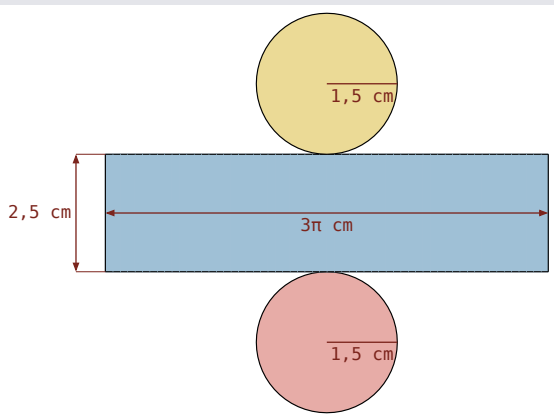
```
// Exemples d'utilisation :
#move(cylunfold(steps: (0, 0.3, 0.6,
1)),dy:1.25cm)
```



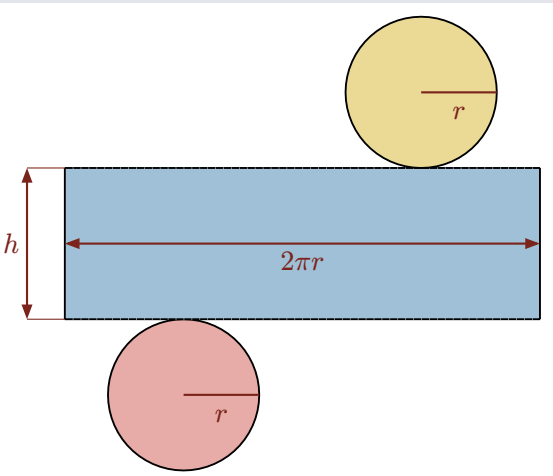
```
// Cotes sur le solide fermé
#cylunfold(t: 0, cotes: true)
```




```
// Patron à plat avec les vraies valeurs
#move(cylunfold(view: "top", cotes: true,
radius: 1.5, height: 2.5, cote-labels:
auto, cote-unit: "cm"),dy:1.5cm)
```



```
// Disques décalés sur le patron (à 25 %
et à 75 % de la longueur du rectangle)
#cylunfold(view: "top", cotes: true, base-
ratio: (0.25, 0.75))
```



Paramètres

```
cylunfold(
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  contact: auto array,
  base-ratio: int float array,
  steps: auto array,
  t: none int float,
  view: str,
  columns: auto int,
  gap: int float,
  labels: none array,
  arrows: bool,
  fill: color,
  base-fill: color,
  top-fill: color,
  segments: int,
  cotes: bool,
  cote-labels: auto dictionary,
  cote-color: color,
  cote-unit: none str,
  cote-digits: int,
  cote-decimal: str,
  edge-stroke: stroke,
  x: array,
  y: array,
  z: array,
  more: content,
  length: length,
  angle: angle
) -> content
```

radius `int` or `float`

Rayon du cylindre

Par défaut: `1`

height `int` or `float`

Hauteur du cylindre

Par défaut: `2`

base `array`

Centre du disque de base, qui reste fixe

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, du disque de base vers l'autre disque (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

contact `auto` or `array`

Direction, dans le plan de la base, du point où le solide touche son patron ; auto : le côté opposé à l'observateur, le patron se déplie alors vers le fond

Par défaut: `auto`

base-ratio `int` or `float` or `array`

Position du ou des disques le long de la surface latérale du patron : ratio entre 0 et 1 mesuré depuis l'extrémité gauche du rectangle en vue de dessus (0.5 : au milieu) ; un nombre pour les deux disques, ou un tableau (bas, haut) pour les placer indépendamment

Par défaut: `0.5`

steps `auto` or `array`

Valeurs du paramètre de dépliage t à afficher côte à côte, de 0 (solide fermé) à 1 (patron à plat). Par défaut (0, 0.5, 1), ou (1,) avec view: « top »

Par défaut: `auto`

t `none` or `int` or `float`

Une seule valeur de t (prioritaire sur steps)

Par défaut: `none`

view `str`

« oblique » : projection oblique x, y, z (par défaut) ; « top » : patron à plat vu de dessus, en vraie grandeur, orienté comme la vue oblique

Par défaut: `"oblique"`

columns `auto` or `int`

Nombre de figures par ligne ; auto : toutes sur une seule ligne

Par défaut: `auto`

gap `int` or `float`

Espace entre les figures (unités du repère)

Par défaut: `0.8`

labels `none` or `array`

Légendes placées sous les figures (une par étape)

Par défaut: `none`

arrows `bool`

Flèches entre deux figures consécutives d'une même ligne

Par défaut: `true`

fill `color`

Couleur des faces

Par défaut: `rgb("#a9cce3")`

base-fill `color`

Couleur du disque fixe

Par défaut: `rgb("#f5b7b1")`

top-fill `color`

Couleur du second disque

Par défaut: `rgb("#f9e79f")`

segments `int`

Finesse du maillage de la surface latérale (nombre de facettes par tour)

Par défaut: `60`

cotes `bool`

Ajoute les cotes : r et h sur le solide fermé (t = 0, vue oblique) ; 2πr, h et r sur le patron (view: « top »)

Par défaut: `false`

cote-labels `auto` or `dictionary`

Étiquettes des cotes : dictionnaire (r:, h:, g:, p:, a:) dont les valeurs remplacent les étiquettes symboliques (la valeur auto donne la vraie valeur), ou auto pour tout remplacer

Par défaut: `(:)`

cote-color `color`

Couleur des cotes

Par défaut: `rgb("#7b241c")`

cote-unit `none` or `str`

Unité ajoutée aux longueurs en mode auto, par exemple « cm »

Par défaut: `none`

cote-digits `int`

Nombre de décimales en mode auto

Par défaut: `2`

cote-decimal `str`

Séparateur décimal en mode auto : « , » ou « . »

Par défaut: `","`

edge-stroke `stroke`

Trait des arêtes

Par défaut: `0.7pt + black`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (vue oblique, comme pour polysection) ; sans effet avec view: « top »

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (vue oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur (vue oblique)

Par défaut: `(-0.5, -0.5)`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas ; la première figure garde ses coordonnées d'origine, les ancres existent pour toutes

Par défaut: `()`

length `length`

Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

angle `angle`

Angle de rotation du dessin en vue:« top »

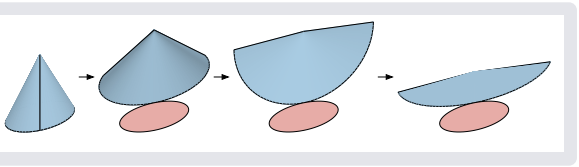
Par défaut: `0deg`

coneunfold

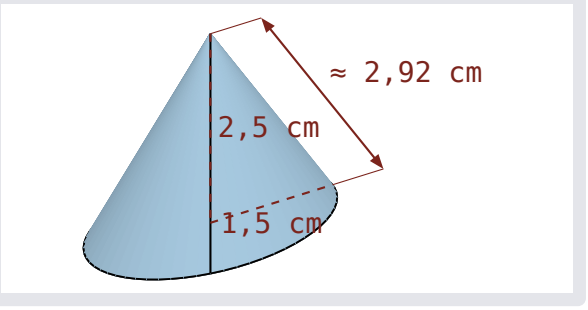
Affiche un cône, son patron et les étapes intermédiaires de son dépliage (CeTZ). Le paramètre d'avancement t va de 0 (solide fermé) à 1 (patron à plat). Le disque de base reste fixe ; la surface latérale se déroule en un secteur circulaire de rayon égal à la génératrice, tangent au disque. Les faces sont opaques, ombrées et triées de la plus lointaine à la plus proche. Avec `cotes: true`, on ajoute les cotes (r, h, g sur le solide fermé ; g, 2πr, angle du secteur et r sur le patron) ; les étiquettes sont à l'extérieur pour les génératrices et sous les lignes pour les autres. Ancres, suffixées −1, −2, … s'il y a plusieurs étapes : « c0 » (centre de la base), « s » (sommet), « pc » (point de contact), « cote-r », « cote-h », « cote-g », « cote-p », « cote-a » (milieux des lignes de cote). Avec `base-ratio`, on choisit

où le disque de base touche l'arc du secteur (ratio mesuré depuis l'extrémité gauche de l'arc en vue de dessus).

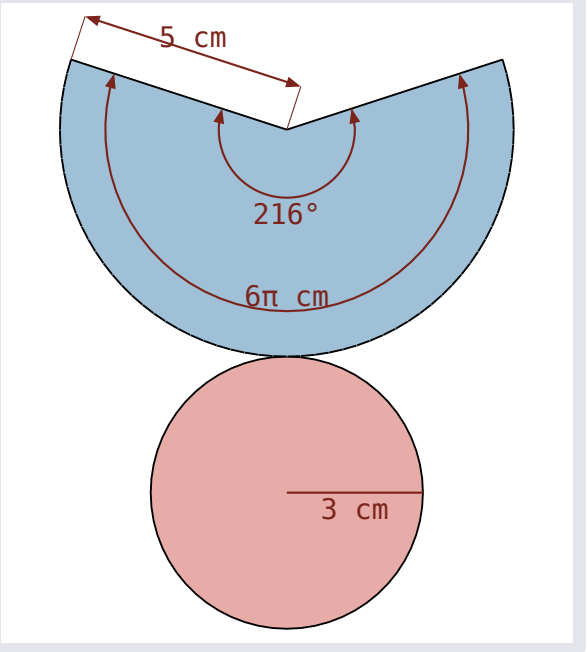
```
// Exemples d'utilisation :
#move(coneunfold(steps: (0, 0.3, 0.6, 1)),
dy:2em)
```



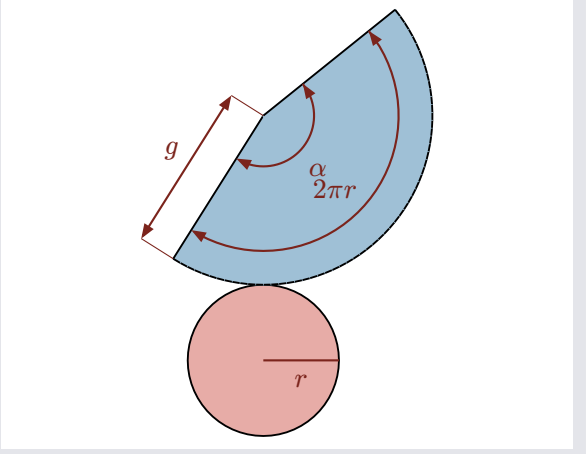
```
// Cotes sur le solide fermé, avec les
vraies valeurs
#coneunfold(t: 0, cotes: true, radius:
1.5, height: 2.5, cote-labels: auto, cote-
unit: "cm")
```




```
// Patron à plat : génératrice, longueur
2πr de l'arc et angle du secteur
#coneunfold(view: "top", cotes: true,
radius: 3, height: 4, cote-labels: auto,
cote-unit: "cm", length: 0.6cm)
```




```
// Disque de base placé à 20 % de l'arc,
depuis la gauche
#coneunfold(view: "top", cotes: true,
base-ratio: 0.2,)
```



Paramètres

```
coneunfold(
  radius: int float,
  height: int float,
  base: array,
  axis: array,
  contact: auto array,
  base-ratio: int float,
  steps: auto array,
  t: none int float,
  view: str,
  columns: auto int,
  gap: int float,
  labels: none array,
  arrows: bool,
  fill: color,
  base-fill: color,
  segments: int,
  cotes: bool,
  cote-labels: auto dictionary,
  cote-color: color,
  cote-unit: none str,
  cote-digits: int,
  cote-decimal: str,
  edge-stroke: stroke,
  x: array,
  y: array,
  z: array,
  more: content,
  length: length,
  angle: angle
) -> content
```

radius `int` or `float`

Rayon du cône

Par défaut: `1`

height `int` or `float`

Hauteur du cône

Par défaut: `2`

base `array`

Centre de la base, qui reste fixe

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, de la base vers le sommet (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

contact `auto` or `array`

Direction, dans le plan de la base, du point où le solide touche son patron ; auto : le côté opposé à l'observateur, le patron se déplie alors vers le fond

Par défaut: `auto`

base-ratio `int` or `float`

Position du disque de base le long de l'arc du secteur : ratio entre 0 et 1 mesuré depuis l'extrémité gauche de l'arc en vue de dessus (0.5 : au milieu)

Par défaut: `0.5`

steps `auto` or `array`

Valeurs du paramètre de dépliage t à afficher côte à côte, de 0 (solide fermé) à 1 (patron à plat). Par défaut (0, 0.5, 1), ou (1,) avec view: « top »

Par défaut: `auto`

t `none` or `int` or `float`

Une seule valeur de t (prioritaire sur steps)

Par défaut: `none`

view `str`

« oblique » : projection oblique x, y, z (par défaut) ; « top » : patron à plat vu de dessus, en vraie grandeur, orienté comme la vue oblique

Par défaut: `"oblique"`

columns `auto` or `int`

Nombre de figures par ligne ; auto : toutes sur une seule ligne

Par défaut: `auto`

gap `int` or `float`

Espace entre les figures (unités du repère)

Par défaut: `0.8`

labels `none` or `array`

Légendes placées sous les figures (une par étape)

Par défaut: `none`

arrows `bool`

Flèches entre deux figures consécutives d'une même ligne

Par défaut: `true`

fill `color`

Couleur des faces

Par défaut: `rgb("#a9cce3")`

base-fill `color`

Couleur de la base

Par défaut: `rgb("#f5b7b1")`

segments `int`

Finesse du maillage de la surface latérale (nombre de facettes par tour)

Par défaut: `60`

cotes `bool`

Ajoute les cotes : r, h et g sur le solide fermé (t = 0, vue oblique) ; g, 2πr, angle du secteur (clé a) et r sur le patron (view: « top »)

Par défaut: `false`

cote-labels `auto` or `dictionary`

Étiquettes des cotes : dictionnaire (r:, h:, g:, p:, a:) dont les valeurs remplacent les étiquettes symboliques (la valeur auto donne la vraie valeur), ou auto pour tout remplacer

Par défaut: `(:)`

cote-color `color`

Couleur des cotes

Par défaut: `rgb("#7b241c")`

cote-unit `none` or `str`

Unité ajoutée aux longueurs en mode auto, par exemple « cm »

Par défaut: `none`

cote-digits `int`

Nombre de décimales en mode auto

Par défaut: `2`

cote-decimal `str`

Séparateur décimal en mode auto : « , » ou « . »

Par défaut: `","`

edge-stroke `stroke`

Trait des arêtes

Par défaut: `0.7pt + black`

x `array`

Image à l'écran du vecteur unitaire de l'axe x (vue oblique, comme pour polysection) ; sans effet avec view: « top »

Par défaut: `(1, 0)`

y `array`

Image à l'écran du vecteur unitaire de l'axe y (vue oblique)

Par défaut: `(0, 1)`

z `array`

Image à l'écran du vecteur unitaire de l'axe z, c'est-à-dire de la profondeur (vue oblique)

Par défaut: `(-0.5, -0.5)`

more `content`

Commandes CeTZ supplémentaires dessinées dans le même canevas ; la première figure garde ses coordonnées d'origine, les ancres existent pour toutes

Par défaut: `()`

length `length`

Échelle du dessin : longueur d'une unité du repère

Par défaut: `1cm`

angle `angle`

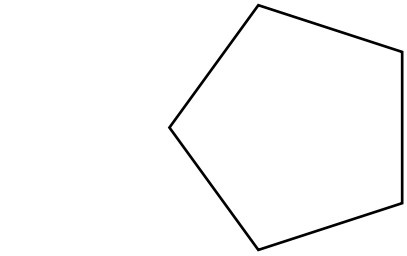
Angle de rotation du dessin en vue:« top »

Par défaut: `0deg`

regpolygon

Sommets d'un polygone régulier dans le plan perpendiculaire à `axis`, dans le sens direct autour de l'axe. Le polygone est défini par son nombre de côtés et par son rayon circonscrit ou la longueur de son côté. Les points sont plans : ils servent à tracer le polygone, mais pas de `vertices` pour un solide.

```
// Exemple d'utilisation :
#cetz.canvas({
  import cetz.draw: *
  line(..regpolygon(5, side: 2, axis: (0,
0, 1)), close: true)
})
```



Paramètres

```
regpolygon(
  n: int,
  radius: none int float,
  side: none int float,
  center: array,
  axis: array,
  phase: int float,
  start: str
) -> array (de points (x, y, z))
```

n `int`

Nombre de côtés (au moins 3)

radius `none` or `int` or `float`

Rayon du cercle (ou de la sphère) circonscrit, c'est-à-dire la distance du centre à un sommet ; donner radius OU side, pas les deux (par défaut radius: 1)

Par défaut: `none`

side `none` or `int` or `float`

Longueur d'un côté (d'une arête pour un solide) ; donner side OU radius

Par défaut: `none`

center `array`

Centre du polygone

Par défaut: `(0, 0, 0)`

axis `array`

Normale au plan du polygone (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

phase `int` or `float`

Rotation supplémentaire autour de l'axe, en degrés, dans le sens direct

Par défaut: `0`

start `str`

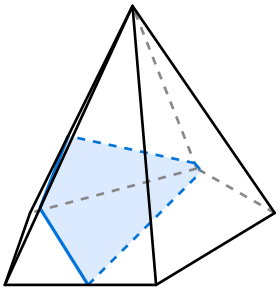
« side » : un côté du polygone de base fait face à l'observateur (direction +z de la vue oblique) ; « vertex » : c'est un sommet

Par défaut: `"side"`

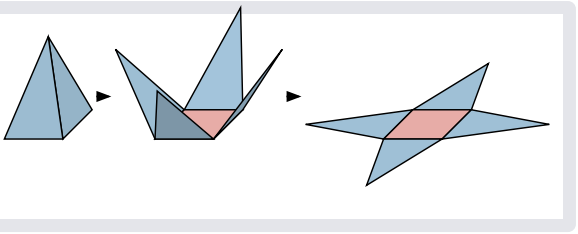
regpyramid

Sommets d'une pyramide régulière : les n sommets de la base (dans le sens direct autour de l'axe), puis le sommet de la pyramide en dernier. À utiliser comme `vertices:` de `polysection` ou de `polyunfold`.

```
// Exemples d'utilisation :
#polysection(1, 1, 0, -0.1, vertices:
regpyramid(5, side: 2, height: 3))
```



```
// Dépliage d'une pyramide à base carrée
#polyunfold(vertices: regpyramid(4, side:
2, height: 3), steps: (0, 0.5, 1), length:
0.5cm)
```



Paramètres

```
regpyramid(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str,
  centered: bool
) -> array (de points (x, y, z))
```

n `int`

Nombre de côtés de la base (au moins 3)

radius `none` or `int` or `float`

Rayon du cercle (ou de la sphère) circonscrit, c'est-à-dire la distance du centre à un sommet ; donner radius OU side, pas les deux (par défaut radius: 1)

Par défaut: none

side `none` or `int` or `float`

Longueur d'un côté (d'une arête pour un solide) ; donner side OU radius

Par défaut: none

height `int` or `float`

Hauteur de la pyramide

Par défaut: 1

center `array`

Centre de la base (ou milieu de la hauteur avec centered: true)

Par défaut: (0, 0, 0)

axis `array`

Direction de l'axe, vers le haut (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

phase `int` or `float`

Rotation supplémentaire autour de l'axe, en degrés, dans le sens direct

Par défaut: `0`

start `str`

« side » : un côté du polygone de base fait face à l'observateur (direction +z de la vue oblique) ; « vertex » : c'est un sommet

Par défaut: `"side"`

centered `bool`

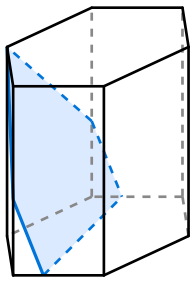
true : `center` est le milieu de la hauteur au lieu du centre de la base

Par défaut: `false`

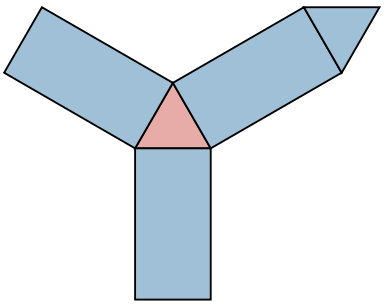
regprism

Sommets d'un prisme droit à base régulière : les n sommets de la base, puis les n sommets de la face supérieure dans le même ordre. À utiliser comme `vertices:` de `polysection` ou de `polyunfold`.

```
// Exemples d'utilisation :
#polysection(1, 0.4, 0, 0.2, vertices:
regprism(6, radius: 1.2, height: 2.5))
```



```
// Patron à plat d'un prisme à base
triangulaire
#polyunfold(vertices: regprism(3, side: 1,
height: 2), view: "top")
```



Paramètres

```
regprism(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str,
  centered: bool
) -> array (de points (x, y, z))
```

n `int`

Nombre de côtés de la base (au moins 3)

radius `none` or `int` or `float`

Rayon du cercle (ou de la sphère) circonscrit, c'est-à-dire la distance du centre à un sommet ; donner radius OU side, pas les deux (par défaut radius: 1)

Par défaut: none

side `none` or `int` or `float`

Longueur d'un côté (d'une arête pour un solide) ; donner side OU radius

Par défaut: none

height `int` or `float`

Hauteur du prisme

Par défaut: 1

center `array`

Centre de la base (ou milieu de la hauteur avec centered: true)

Par défaut: (0, 0, 0)

axis `array`

Direction de l'axe, vers le haut (pas besoin qu'elle soit unitaire)

Par défaut: (0, 1, 0)

phase `int` or `float`

Rotation supplémentaire autour de l'axe, en degrés, dans le sens direct

Par défaut: `0`

start `str`

« side » : un côté du polygone de base fait face à l'observateur (direction +z de la vue oblique) ; « vertex » : c'est un sommet

Par défaut: `"side"`

centered `bool`

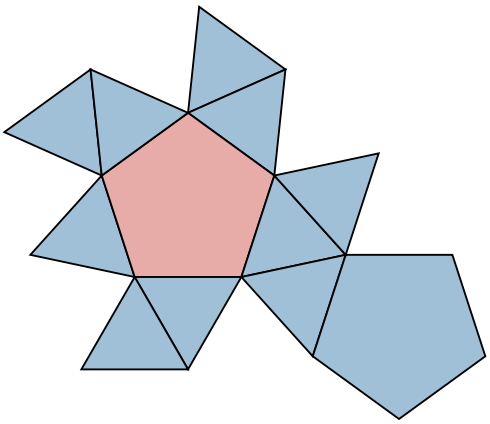
true : `center` est le milieu de la hauteur au lieu du centre de la base

Par défaut: `false`

regantiprism

Sommets d'un antiprisme : comme un prisme, mais la face supérieure est tournée de 180°/n par rapport à la base, ce qui donne des faces latérales triangulaires. Mêmes conventions que `regprism`.

```
// Exemple d'utilisation :
#polyunfold(vertices: regantiprism(5,
radius: 1.2, height: 1.2), view: "top")
```



Paramètres

```
regantiprism(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str,
  centered: bool
) -> array (de points (x, y, z))
```

n `int`

Nombre de côtés de chaque base (au moins 3)

radius `none` or `int` or `float`

Rayon du cercle (ou de la sphère) circonscrit, c'est-à-dire la distance du centre à un sommet ; donner radius OU side, pas les deux (par défaut radius: 1)

Par défaut: `none`

side `none` or `int` or `float`

Longueur d'un côté (d'une arête pour un solide) ; donner side OU radius

Par défaut: `none`

height `int` or `float`

Distance entre les plans des deux bases

Par défaut: `1`

center `array`

Centre de la base (ou milieu de la hauteur avec centered: true)

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, vers le haut (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

phase `int` or `float`

Rotation supplémentaire autour de l'axe, en degrés, dans le sens direct

Par défaut: `0`

start `str`

« side » : un côté du polygone de base fait face à l'observateur (direction +z de la vue oblique) ; « vertex » : c'est un sommet

Par défaut: `"side"`

centered `bool`

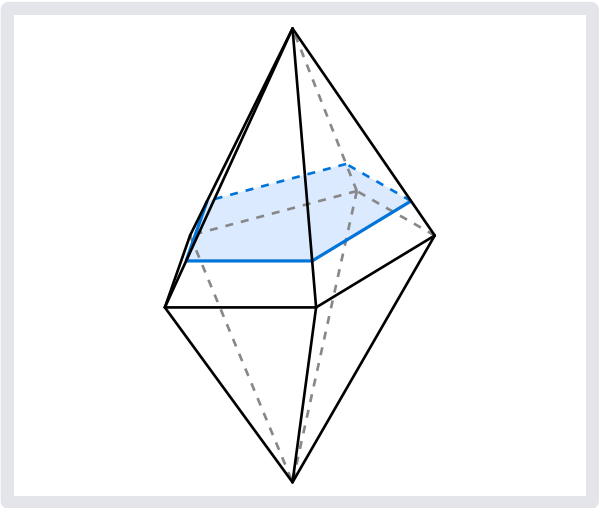
true : `center` est le milieu de la hauteur au lieu du centre de la base

Par défaut: `false`

regbipyramid

Sommets d'une bipyramide : les n sommets du polygone du milieu, puis le sommet du haut, puis celui du bas.

```
// Exemple d'utilisation :
#polysection(0, 1, 0, -0.5, vertices:
regbipyramid(5, side: 2, height: 3))
```



Paramètres

```
regbipyramid(
  n: int,
  radius: none int float,
  side: none int float,
  height: int float,
  center: array,
  axis: array,
  phase: int float,
  start: str
) -> array (de points (x, y, z))
```

n `int`

Nombre de côtés du polygone du milieu (au moins 3)

radius `none` or `int` or `float`

Rayon du cercle (ou de la sphère) circonscrit, c'est-à-dire la distance du centre à un sommet ; donner radius OU side, pas les deux (par défaut radius: 1)

Par défaut: `none`

side `none` or `int` or `float`

Longueur d'un côté (d'une arête pour un solide) ; donner side OU radius

Par défaut: `none`

height `int` or `float`

Hauteur de chaque pyramide (distance du polygone du milieu à chaque sommet)

Par défaut: `1`

center `array`

Centre du polygone du milieu

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, vers le haut (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

phase `int` or `float`

Rotation supplémentaire autour de l'axe, en degrés, dans le sens direct

Par défaut: `0`

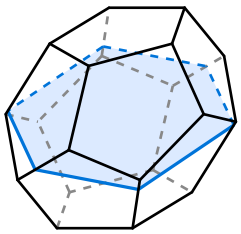
start `str`

« side » : un côté du polygone de base fait face à l'observateur (direction +z de la vue oblique) ; « vertex » : c'est un sommet

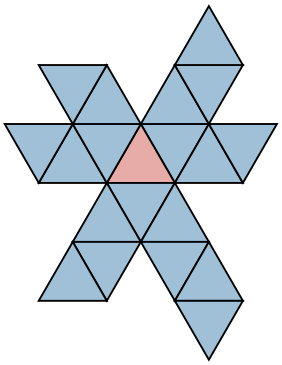
Par défaut: `"side"`

regpolyhedron
Sommets d'un polyèdre régulier (solide de Platon) : « tétraèdre », « cube », « octaèdre », « dodécaèdre » ou « icosaèdre » (les noms anglais sont acceptés). Le solide est posé sur l'une de ses faces, l'axe pointant vers le haut, et un côté de cette face fait face à l'observateur ; c'est aussi la face que `polyunfold` laisse fixe par défaut.

```
// Exemples d'utilisation :
#polysection(0.3, 1, 0.2, 0, vertices:
regpolyhedron("dodécaèdre", side: 1))
```



```
// Patron à plat d'un icosaèdre
#polyunfold(vertices:
regpolyhedron("icosaèdre", side: 1.5),
view: "top", length: 0.6cm)
```



Paramètres

```
regpolyhedron(
  name: str,
  radius: none int float,
  side: none int float,
  center: array,
  axis: array,
  phase: int float
) -> array (de points (x, y, z))
```

name `str`

Nom du polyèdre : « tétraèdre », « cube », « octaèdre », « dodécaèdre » ou « icosaèdre »

radius `none` or `int` or `float`

Rayon du cercle (ou de la sphère) circonscrit, c'est-à-dire la distance du centre à un sommet ; donner radius OU side, pas les deux (par défaut radius: 1)

Par défaut: none

side `none` or `int` or `float`

Longueur d'un côté (d'une arête pour un solide) ; donner side OU radius

Par défaut: none

center `array`

Centre du solide

Par défaut: `(0, 0, 0)`

axis `array`

Direction de l'axe, vers le haut (pas besoin qu'elle soit unitaire)

Par défaut: `(0, 1, 0)`

phase `int` or `float`

Rotation supplémentaire autour de l'axe, en degrés, dans le sens direct

Par défaut: `0`